

Table des Matieres

1. Vision Generale d'AgentForge	4
1.1 Philosophie : 1 Kernel, 7 Configurations	4
1.2 Les 7 Agents Produit	4
1.3 Architecture Macroscopique	5
2. Le MoA Kernel : Coeur de l'Orchestration	5
2.1 Mixture-of-Agents (MoA) : 9 LLMs en Parallele	5
2.2 Flux d'Orchestration en 4 Etapes	6
2.3 SuperAgentOrchestrator : Implementation	6
3. DAG-Based Code Generation	7
3.1 Principe du DAG de Dependances	7
3.2 DAG par Type d'Agent	8
3.3 Context-Aware Generation	8
4. Reflection Agent : Le Mecanisme Secret	8
4.1 Vote Multi-Criteres Ponderes	8
4.2 Algorithme de Synthese	9
5. 3-Level Auto-Fix Engine	9
5.1 Cascade de Correction en 3 Niveaux	9

6. Infrastructure Partagee	10
6.1 Cache Manager (L1 + L2)	10
6.2 Cost Optimizer et Routeur RL	11
6.3 Sandbox Manager (Docker Warm Pool)	11
6.4 Cloudflare Edge Deployer	11
7. Agent Registry et Systeme de Configuration	12
7.1 Configuration Dynamique par Agent	12
7.2 Ajouter un Nouvel Agent	12
8. Securite et Rate Limiting	13
8.1 Architecture de Securite Multi-Couches	13
8.2 Rate Limiting (Token Bucket)	13
9. Schema de Base de Donnees	14
9.1 Tables Principales	14
10. Feuille de Route de Production	14
10.1 Phase 1 : Fondations (Semaines 1-3)	14
10.2 Phase 2 : Agent Developpeur (Semaines 4-6)	14
10.3 Phase 3 : Agent SLIDES (Semaines 7-9)	15
10.4 Phase 4 : Agents DOC + DATA + RECHERCHE (Semaines 10-14)	15
10.5 Phase 5 : Scale + EMAIL + MARKETING (Semaines 15-18)	15

11. Estimation des Couts et Modele Economique	16
11.1 Couts de Construction	16
11.2 Modele Economique et Rentabilite	16
12. Les 12 Mecanismes d'Orchestration	17
12.1 Vue d'Ensemble des 12 Mecanismes	17
12.2 Interactions entre Mecanismes	18
13. Les 25 Commandements de l'Architecture Agent	18
14. Prochaines Etapes et Recommandations	19
14.1 Actions Immmediates	19
14.2 Risques et Mitigations	19
14.3 Indicateurs de Succes	20

1. Vision Generale d'AgentForge

1.1 Philosophie : 1 Kernel, 7 Configurations

AgentForge repose sur un principe fondamental qui distingue radicalement cette plateforme de toute approche naive de construction d'agents IA : au lieu de developper sept agents independants avec chacun leur propre logique, leur propre orchestration et leur propre infrastructure, nous construisons un **noyau unique** (le Kernel MoA) que sept configurations qui parametrent. Ce choix architectural transforme la maintenance, les couts et la scalabilite du systeme entier. Chaque agent n'est pas un code separe, mais une configuration specialisee du meme moteur d'orchestration.

Concretement, cela signifie que l'Agent Developpeur, l'Agent SLIDES, l'Agent DOC, l'Agent DATA, l'Agent RECHERCHE, l'Agent EMAIL et l'Agent MARKETING partagent tous le meme SuperAgentOrchestrator, le meme ReflectionAgent, le meme CostOptimizer et le meme CacheManager. Seuls trois elements varient d'un agent a l'autre : la definition du DAG de generation, les criteres de reflexion ponderes, et le routage des modeles. Cette approche elimine la duplication de code, centralise la maintenance, et permet d'ajouter un nouvel agent en quelques jours plutot que quelques semaines.

1.2 Les 7 Agents Produit

Agent	Role Principal	DAG	Modele Principal
Developpeur	Generation code, APIs, deployment	Config > Types > API > UI	Claude-3.7-Sonnet
SLIDES	Presentations, design, mise en page	Template > Layout > Content > Design	GPT-4o
DOC	Rapports PDF/DOCX, documents longs	Outline > Sections > Content > PDF	Claude-3.7-Sonnet
DATA	Analyse de donnees, visualisations	Schema > Query > Process > Visual	DeepSeek-R1
RECHERCHE	Synthese multi-sources, veille	Query > Search > Extract > Synthesize	GPT-4o
EMAIL	Redaction, personnalisation, A/B test	Context > Draft > Refine > Send	GPT-4o-Mini
MARKETING	Copy, funnels, SEO, analytics	Brief > Copy > Visual > Funnel	Claude-3.7-Sonnet

Tableau 1 : Les 7 agents produit et leurs specialisations

1.3 Architecture Macroscopique

L'architecture d'AgentForge se decompose en quatre couches horizontales clairement separees. La couche superieure est le Dashboard Utilisateur (React + Vite), qui fournit l'interface de pilotage de tous les agents. Vient ensuite l'API Gateway (Hono + JWT), qui gere l'authentification, le rate limiting et le routage vers l'Agent Registry. L'Agent Registry est le routeur dynamique qui identifie quel agent est sollicite et charge sa configuration specifique. Enfin, la couche inferieure est le MoA Kernel, le coeur du systeme, qui orchestre les 9 LLMs en parallele, applique la reflexion multi-criteres, et gere l'auto-correction en cascade.

L'infrastructure partagee soutient l'ensemble du systeme avec un Cache Manager (L1 in-memory + L2 Redis), un Cost Optimizer avec routeur RL, un Sandbox Manager (Docker Warm Pool), un CloudflareDeployer pour les deploiements en moins de 60 secondes, une base PostgreSQL pour les donnees persistantes, et un systeme d'analytics avec entrainement RL continu. Cette separation stricte des responsabilites garantit que chaque couche peut etre mise a l'echelle independamment.

2. Le MoA Kernel : Coeur de l'Orchestration

2.1 Mixture-of-Agents (MoA) : 9 LLMs en Parallele

Le Mixture-of-Agents est le premier mecanisme secret qui distingue AgentForge d'une chaine LLM sequentielle classique. Au lieu d'envoyer une requete a un seul modele et d'esperer le meilleur resultat, le MoA decompose la tache en sous-taches, route chaque sous-tache vers le modele le plus adapte, execute toutes les sous-taches en parallele, puis synthetise les meilleurs elements de chaque sortie. Ce processus en quatre etapes (Decomposition, Routage, Execution Parallele, Synthese) est la cle de la qualite exceptionnelle des resultats obtenus.

Les neuf modeles utilises sont : Claude-3.7-Sonnet (code, architecture), GPT-4o (raisonnement general, contenu), DeepSeek-R1 (logique, mathematiques), Gemini-2.5-Pro (contexte long, analyse), GPT-o1 (raisonnement avance), Llama-3.3 (generation rapide), Qwen-2.5 (multilingue), Mistral-Large (texte structure), et Gemini-2.0-Flash (taches legerees ultra-rapides). Chaque modele recoit uniquement les sous-taches pour lesquelles il excelle, ce qui maximise la qualite tout en optimisant les couts.

Modele	Specialite	Cout/1K tokens	Vitesse
Claude-3.7-Sonnet	Code, architecture, raisonnement	\$0.003	Moyen
GPT-4o	Contenu general, multimodal	\$0.005	Rapide
DeepSeek-R1	Logique, mathematiques, raisonnement	\$0.001	Lent

Gemini-2.5-Pro	Contexte long, analyse documentaire	\$0.003	Moyen
GPT-o1	Raisonnement avance, planification	\$0.015	Lent
Llama-3.3-70B	Generation rapide, open-source	\$0.0006	Tres rapide
Qwen-2.5-72B	Multilingue, chinois	\$0.0005	Rapide
Mistral-Large	Texte structure, synthese	\$0.002	Moyen
Gemini-2.0-Flash	Taches legeres, ultra-rapide	\$0.0001	Tres rapide

Tableau 2 : Les 9 modeles LLM et leurs specialites

2.2 Flux d'Orchestration en 4 Etapes

Le flux complet d'orchestration suit un pipeline rigoureux en quatre etapes. Premierement, la **Decomposition** : le SuperAgentOrchestrator analyse la requete utilisateur et la decompose en sous-taches independantes. Par exemple, "Cree une application e-commerce" serait decompose en : generation du schema de base de donnees, creation des types TypeScript, implementation des routes API, construction des composants React, et configuration du deploiement. Deuxiemement, le **Routage** : chaque sous-tache est affectee au modele le plus adapte selon une table de routage optimisee par apprentissage par renforcement. Troisiemement, l'**Execution Parallele** : toutes les sous-taches sont lancees simultanement, reduisant drastiquement le temps de reponse total. Quatriemement, la **Synthese** : le ReflectionAgent evalue les sorties de chaque modele selon des criteres ponderes et construit le resultat final en fusionnant les meilleurs elements.

2.3 SuperAgentOrchestrator : Implementation

L'orchestrateur central est implemente en TypeScript et gere l'ensemble du cycle de vie d'une requete. Il maintient la configuration des 9 modeles, la table de routage par type de sous-tache, et les seuils de confiance. Lorsqu'une requete arrive, il applique la decomposition via Claude-3.7-Sonnet (le meilleur modele pour cette tache structuree), puis distribue les sous-taches aux modeles selectionnes en parallele via Promise.all. Apres execution, il transmet les resultats au ReflectionAgent pour evaluation. Si le score de confiance est inferieur a 0.95, il declenche un cycle de raffinement qui re-injecte les feedbacks dans les modeles pour une seconde passe.

Etape	Composant	Modele Utilise	Sortie
1. Decomposition	SuperAgentOrchestrator	Claude-3.7-Sonnet	Liste de sous-taches
2. Routage	CostOptimizer + RL Router	Table de routage	Mapping tache-modele

3. Execution	9 LLMs en parallele	Selon routage	9 sorties brutes
4. Synthese	ReflectionAgent	GPT-4o + vote	Resultat fusionne

Tableau 3 : Pipeline d'orchestration en 4 etapes

3. DAG-Based Code Generation

3.1 Principe du DAG de Dependances

La generation de code basee sur DAG (Directed Acyclic Graph) est le deuxieme mecanisme secret qui rend AgentForge si performant. Plutot que de generer tous les fichiers sequentiellement, le systeme construit un graphe de dependances entre les fichiers a generer, effectue un tri topologique, puis genere les fichiers niveau par niveau. Les fichiers du meme niveau (sans dependances entre eux) sont generes en parallele, ce qui accelere considerablement le processus. Ce mecanisme est crucial pour la coherence inter-fichiers : chaque fichier recoit en contexte le code des fichiers de ses dependances deja generees.

Pour l'Agent Developpeur, le DAG standard comporte quatre niveaux. Le Niveau 0 (Configuration) contient les fichiers de configuration globaux (tsconfig, package.json, env). Le Niveau 1 (Types et Schema) genere les interfaces TypeScript et le schema Prisma. Le Niveau 2 (API et Auth) cree les routes API et le middleware d'authentification, en utilisant les types du Niveau 1 comme contexte. Enfin, le Niveau 3 (Components) construit les composants React, en se basant sur les types et l'API des niveaux precedents. Cette approche garantit que chaque fichier est genere avec une connaissance complete de ses dependances.

Niveau	Fichiers	Dependances	Modele Recommande
L0 - Config	tsconfig, package.json, .env	Aucune	Gemini-2.0-Flash
L1 - Types	Interfaces TypeScript, Schema Prisma	L0	Claude-3.7-Sonnet
L2 - API/Auth	Routes API, Middleware, Services	L0 + L1	Claude-3.7-Sonnet
L3 - Components	Composants React, Pages, Hooks	L0 + L1 + L2	GPT-4o

Tableau 4 : Niveaux du DAG pour l'Agent Developpeur

3.2 DAG par Type d'Agent

Chaque agent possède son propre DAG spécialisé. L'Agent SLIDES utilise un DAG en quatre niveaux : Template (sélection du modèle de présentation) > Layout (structure des diapositives) > Content (génération du contenu texte et visuel) > Design (application du style, couleurs, typographie). L'Agent DOC suit : Outline > Sections > Content > PDF/DOCX. L'Agent DATA : Schema > Query > Process > Visual. L'Agent RECHERCHE : Query > Search > Extract > Synthesize. L'Agent EMAIL : Context > Draft > Refine > Send. Et l'Agent MARKETING : Brief > Copy > Visual > Funnel. Chaque DAG est défini par une simple structure JSON que le CodeGenerator interprète et exécute.

3.3 Context-Aware Generation

Le mécanisme de génération contextuelle est un élément crucial souvent omis dans les implémentations naïves. Lorsqu'un fichier est généré au Niveau N, il reçoit en entrée non seulement le prompt initial de l'utilisateur, mais aussi le contenu complet de tous les fichiers générés aux niveaux 0 à N-1 dont il dépend directement. Cette approche, appelée Linear Context Management, évite les conflits de contexte en n'injectant que les dépendances directes plutôt que l'intégralité du code généré. Par exemple, un composant React au Niveau 3 reçoit les types du Niveau 1 et l'API du Niveau 2, mais pas les fichiers de configuration du Niveau 0. Cela permet de rester dans la fenêtre de tokens des modèles tout en garantissant la cohérence du code généré.

4. Reflection Agent : Le Mécanisme Secret

4.1 Vote Multi-Critères Pondérés

Le Reflection Agent est sans doute le mécanisme le plus important et le moins évident de toute l'architecture. C'est la clé du score GAIA de 87.8% atteint par Genspark. Le principe est simple mais dévotement efficace : au lieu d'accepter la sortie d'un seul modèle, le Reflection Agent fait évaluer la sortie de chaque modèle par plusieurs autres modèles selon six critères pondérés, puis synthétise le meilleur de chaque évaluation pour produire un résultat final supérieur à n'importe quelle sortie individuelle.

Les six critères de réflexion et leurs pondérations varient selon l'agent. Pour l'Agent Développeur, les critères sont : Qualité du Code (30%), Fonctionnalité (25%), Performance (15%), Sécurité (15%), Expérience Utilisateur (10%) et Tests (5%). La somme des pondérations est toujours 100%. Chaque critère reçoit un score entre 0 et 1 de chaque modèle évaluateur, et le score final est la moyenne pondérée. Si le score de confiance final est inférieur au seuil de 0.95, un cycle de raffinement est déclenché.

Critere	Dev	SLIDES	DOC	DATA
Qualite / Design	30%	30%	30%	35%
Fonctionnalite / Contenu	25%	25%	25%	25%
Performance / Coherence	15%	20%	20%	15%
Securite / Style	15%	15%	10%	5%
UX / Precision	10%	5%	10%	15%
Tests / Visualisation	5%	5%	5%	5%
	0.95	0.92	0.93	0.94

Tableau 5 : Criteres de reflexion ponderes par agent

4.2 Algorithme de Synthese

L'algorithme de synthese du Reflection Agent opere en trois phases. La premiere phase est l'**Evaluation Individuelle** : chaque sortie de modele est evaluee par trois modeles evaluateurs differents sur les six criteres ponderes. Les evaluateurs typiques sont Claude-3.7-Sonnet, GPT-4o et Gemini-2.5-Pro. La deuxieme phase est l'**Aggregation** : pour chaque critere, le systeme selectionne la meilleure sortie parmi tous les modeles generateurs. Par exemple, si le modele A produit le meilleur code (score qualite 0.92) mais le modele B offre la meilleure securite (score securite 0.88), le systeme extrait la qualite du modele A et la securite du modele B. La troisieme phase est la **Fusion** : un modele de synthese (generalement GPT-4o ou Claude-3.7-Sonnet) prend les meilleurs extraits de chaque critere et les fusionne en un resultat coherent et unifie.

Ce processus iteratif est la raison pour laquelle le score de confiance de 0.95 est atteint. Si apres la premiere synthese le score est inferieur au seuil, le systeme re-injecte les feedbacks detailles dans les modeles generateurs avec des instructions precises d'amelioration, et le cycle se repete. En pratique, la plupart des requetes atteignent le seuil en 1 a 2 iterations, ce qui maintient un bon equilibre entre qualite et temps de reponse.

5. 3-Level Auto-Fix Engine

5.1 Cascade de Correction en 3 Niveaux

Le moteur d'auto-correction en trois niveaux est le mecanisme qui garantit que le code genere fonctionne reellement, pas seulement qu'il semble correct. Quand une erreur est detectee (par exemple lors de l'execution dans le sandbox), le systeme tente d'abord une correction par pattern, puis par analyse AST,

et enfin par correction LLM. Cette cascade est optimisee pour minimiser les couts : les corrections les moins cheres sont essayees en premier, et les plus couteuses ne sont utilisees qu'en dernier recours.

Le **Niveau 1 - Pattern-Based Fix** (taux de reussite 40%) fonctionne par reconnaissance de motifs d'erreur courants. Il maintient un dictionnaire de plus de 200 patterns d'erreur avec leurs corrections associees. Par exemple, "Cannot read property of undefined" est corrige en ajoutant un optional chaining, ou "Module not found" est corrige en ajoutant l'import manquant. Ce niveau est quasi instantane et ne coute rien en tokens LLM.

Le **Niveau 2 - AST-Based Fix** (taux de reussite 30% supplementaires) analyse le code genere sous forme d'arbre syntaxique abstrait pour identifier les erreurs structurelles : types incompatibles, variables non declarees, signatures de fonction incorrectes. Ce niveau utilise un parseur TypeScript/Python pour naviguer dans l'AST et appliquer des transformations ciblees. Il est plus lent que le Niveau 1 mais reste gratuit en termes de couts LLM.

Le **Niveau 3 - LLM-Based Fix** (taux de reussite 25% supplementaires) envoie le code errone, le message d'erreur et le contexte complet a un LLM (generalement Claude-3.7-Sonnet ou GPT-4o) avec des instructions de correction precise. Ce niveau est le plus couteux mais aussi le plus puissant, capable de corriger des erreurs logiques complexes que les deux premiers niveaux ne peuvent pas detecter. Les 5% restants correspondent aux cas ou la regeneration complete du fichier est necessaire.

Niveau	Methode	Taux de reussite	Cout	Temps moyen
1 - Pattern	Dictionnaire de 200+ patterns	40%	\$0	< 100ms
2 - AST	Analyse syntaxique abstraite	30%	\$0	500ms - 2s
3 - LLM	Correction par modele de langage	25%	\$0.01 - \$0.05	3 - 10s
Regeneration	Regeneration complete du fichier	5%	\$0.05 - \$0.15	10 - 30s

Tableau 6 : Cascade de correction en 3 niveaux

6. Infrastructure Partagee

6.1 Cache Manager (L1 + L2)

Le Cache Manager implemente un systeme de cache a deux niveaux qui reduit drastiquement les couts et les temps de reponse. Le **Cache L1** est un cache in-memory avec une capacite de 100 entrees et un

TTL de 1 minute. Il est utilise pour les requetes repetees au sein d'une meme session, comme les definitions de types ou les schemas de base de donnees. Le **Cache L2** est un cache Redis avec une capacite illimitee et un TTL configurable (par default 1 heure). Il persiste entre les sessions et est partage entre toutes les instances de l'API.

Les cle de cache sont generees par hashage SHA256 du prompt combine avec la configuration de l'agent et le contexte. L'invalidation du cache suit un systeme de patterns : lorsqu'un fichier est regenere, tous les caches qui dependent de ce fichier sont invalides. Ce mecanisme garantit que le cache ne sert jamais de contenu obsolete tout en maximisant le taux de hit. En production, le taux de hit du cache atteint typiquement 30 a 45%, ce qui represente des economies significatives sur les couts LLM.

6.2 Cost Optimizer et Routeur RL

Le Cost Optimizer est le composant qui equilibre qualite, cout et vitesse pour chaque sous-tache. Il utilise une formule de scoring pondere : $\text{Score} = \text{Qualite} * 0.50 + (1 - \text{Cout}/\text{Cout_max}) * 0.30 + (1 - \text{Temps}/\text{Temps_max}) * 0.20$. Pour chaque sous-tache, il evalue tous les modeles capables de la traiter, calcule leur score, et selectionne celui avec le meilleur ratio. Si le budget utilisateur est limite, il filtre les modeles dont le cout depasse le budget restant et selectionne le meilleur parmi les modeles restants.

Le Routeur RL (Reinforcement Learning) va plus loin en apprenant des interactions passees. Il collecte des donnees d'entrainement dans la table `rl_training_data` : pour chaque sous-tache executee, il enregistre le modele utilise, le score de qualite obtenu, le cout reel et le temps de reponse. Un modele d'apprentissage par renforcement est entraine periodiquement sur ces donnees pour affiner la politique de routage. Avec le temps, le routeur apprend quel modele est le plus adapte a chaque type de sous-tache, aux habitudes specifiques de chaque utilisateur, et aux patterns d'utilisation globaux. C'est un mecanisme d'amelioration continue qui rend la plateforme plus efficace avec le temps.

6.3 Sandbox Manager (Docker Warm Pool)

Le Sandbox Manager gere un pool de conteneurs Docker prechauffes pour l'execution isolee du code genere. Plutot que de creer un nouveau conteneur pour chaque requete (ce qui prendrait 5 a 10 secondes), le systeme maintient un pool de 5 conteneurs toujours en etat "pret". Quand une requete arrive, un conteneur est alloue, le code y est injecte, execute, et le resultat est recupere. Le conteneur est ensuite nettoye et remis dans le pool. Les limites de ressources sont strictes : 80% de CPU maximum, 8 Go de RAM, et un timeout de 30 secondes. L'isolation reseau est assuree par des regles iptables qui n'autorisent que les connexions sortantes HTTP/HTTPS. Les commandes dangereuses (`rm -rf`, `sudo`, `chmod 777`, etc.) sont bloques par une liste noire.

6.4 Cloudflare Edge Deployer

Le Cloudflare Deployer automatise le deploiement des applications generees en moins de 60 secondes. Le processus se decompose en six etapes : creation du projet Pages, upload des fichiers, provisioning de

la base D1, execution des migrations, configuration des variables d'environnement, et attribution du domaine. L'avantage de Cloudflare est la distribution edge : l'application est deployee sur plus de 300 datacenters dans le monde, garantissant des temps de reponse inferieurs a 50ms partout. Le cout est egalement extremement competitif : le plan gratuit offre 500 requetes/mois, et le plan payant commence a \$5/mois pour 100 000 requetes.

7. Agent Registry et Systeme de Configuration

7.1 Configuration Dynamique par Agent

L'Agent Registry est le routeur dynamique qui permet d'ajouter de nouveaux agents sans modifier le code du kernel. Chaque agent est defini par une configuration JSON qui specifie cinq elements : l'identifiant et le nom de l'agent, la definition du DAG de generation (niveaux, dependances, fichiers types), les criteres de reflexion ponderes, la table de routage des modeles (quel modele pour quel type de sous-tache), et le pipeline de sortie (format, post-traitement, livraison). Quand une requete arrive, l'Agent Registry identifie l'agent cible, charge sa configuration, et passe le relais au MoA Kernel qui execute le pipeline avec les parametres de cet agent.

Parametre	Dev	SLIDES	DOC
DAG Niveaux	Config > Types > API > UI	Template > Layout > Content > Design	Outline > Sections > Content > PDF
Critere Principal	Qualite 30%	Design 30%	Structure 30%
Modele Principal	Claude-3.7-Sonnet	GPT-4o	Claude-3.7-Sonnet
Sandbox	Oui (Docker)	Non	Non
Deployer	Cloudflare Pages	N/A	Telechargement direct
Format Sortie	Code source + URL	PPTX / PDF	PDF / DOCX
Seuil Confiance	0.95	0.92	0.93

Tableau 7 : Comparaison des configurations par agent

7.2 Ajouter un Nouvel Agent

L'ajout d'un nouvel agent se resume a la creation d'un fichier de configuration JSON et a l'implementation d'eventuels post-traitements specifiques. Le processus complet prend typiquement 2 a 3 jours : jour 1 pour la definition du DAG et des criteres de reflexion, jour 2 pour le reglage fin du

routage des modeles et les tests, et jour 3 pour l'integration dans le dashboard et la documentation. Aucune modification du kernel n'est necessaire, ce qui elimine le risque de regression sur les agents existants. Cette extensibilite est la consequence directe de l'architecture "1 Kernel, 7 Configurations" : le kernel est stable et eprouve, seules les configurations evoluent.

8. Securite et Rate Limiting

8.1 Architecture de Securite Multi-Couches

La securite d'AgentForge repose sur six couches de protection complementaires. La premiere couche est la **Validation Zod** : toutes les entrees utilisateur sont validees par des schemas Zod stricts avant traitement. La deuxieme couche est les **Requetes Parametrees** : toutes les requetes SQL utilisent des parametres bindes pour prevenir les injections SQL. La troisieme couche est la **Sanitization XSS** : tout contenu genere par les LLM est sanize avant affichage. La quatrieme couche est le **CORS** : seuls les domaines autorises peuvent acceder a l'API. La cinquieme couche est l'**Authentification JWT** : chaque requete est authentifiee par un token JWT avec rotation automatique. La sixieme couche est la **Securite du Sandbox** : les commandes dangereuses sont bloques, l'accès réseau est restreint, et les ressources sont limitees.

8.2 Rate Limiting (Token Bucket)

Le rate limiting est implemente via un algorithme de Token Bucket avec un script Lua execute dans Redis pour garantir l'atomicite. Quatre niveaux de taux sont definis selon le plan utilisateur. Le plan Gratuit permet 10 requetes par heure, le plan Plus offre 50 requetes par heure, le plan Pro monte a 200 requetes par heure, et le plan Entreprise offre 1000 requetes par heure. Le script Lua verifie le nombre de tokens restants, consomme un token si disponible, et retourne le nombre restant avec le temps avant rechargement. Si le bucket est vide, la requete est rejete avec un code 429 et un header Retry-At indiquant quand le prochain token sera disponible.

Plan	Requetes/heure	Prix/mois	Cout LLM estime
Gratuit	10	\$0	N/A
Plus	50	\$29	\$15-20
Pro	200	\$99	\$50-70
Entreprise	1000	\$299	\$150-250

Tableau 8 : Plans et rate limiting

9. Schema de Base de Donnees

9.1 Tables Principales

Le schema de base de donnees est construit sur PostgreSQL 16 avec sept tables principales. La table **users** stocke les informations utilisateur avec authentification JWT et gestion des plans. La table **projects** gere les projets avec configuration JSONB flexible et statut de deploiement. La table **generation_sessions** enregistre chaque session de generation avec le prompt, le modele utilise, le score de qualite et les metriques de cout et temps. La table **rl_training_data** collecte les donnees d'entrainement pour le routeur RL. La table **error_recovery_log** trace les corrections appliquees par l'Auto-Fix Engine. La table **cost_tracking** suit les couts par utilisateur, modele et jour. Enfin, la table **analytics_events** capture tous les evenements pour le dashboard analytique.

Les index sont optimises pour les requetes les plus frequentes : index composites sur (user_id, created_at) pour les sessions de generation, index sur (model, task_type) pour les donnees RL, et index GIN sur les colonnes JSONB pour les recherches dans les configurations. Les foreign keys sont configurees avec ON DELETE CASCADE pour les donnees dependantes et ON DELETE SET NULL pour les references optionnelles.

10. Feuille de Route de Production

10.1 Phase 1 : Fondations (Semaines 1-3)

La premiere phase pose les bases du systeme. La Semaine 1 est consacree au MoA Kernel et a l'Agent Registry : implementation du SuperAgentOrchestrator avec configuration des 9 modeles, systeme de configuration dynamique par agent, et tests unitaires du kernel. La Semaine 2 construit le Reflection Engine et l'Auto-Fix : implementation du ReflectionAgent avec criteres configurables, seuil de confiance ajustable par agent, 3-Level Auto-Fix, et tests d'evaluation qualite. La Semaine 3 deploye l'infrastructure partagee : API Gateway avec Hono, PostgreSQL et Redis avec migrations, CacheManager L1+L2, CostOptimizer et RL Router, et squelette du Dashboard React.

10.2 Phase 2 : Agent Developpeur (Semaines 4-6)

La deuxieme phase construit l'agent phare. La Semaine 4 implemente le DAG Code Generation : DAG Builder pour fichiers code, generation niveau par niveau (Config, Types, API, UI), Context-Aware Generation et Linear Context Management. La Semaine 5 ajoute le Sandbox et l'Auto-Fix specialise code : SandboxManager avec Docker Warm Pool, execution isolee avec limites de ressources, Auto-Fix pour erreurs TypeScript, React, Python, et tests de bout en bout. La Semaine 6 finalise le deploiement et l'integration : CloudflareDeployer, pipeline complet du prompt au deploiement, dashboard avec historique projets, couts et metriques. A la fin de cette phase, l'Agent Developpeur est le premier MVP

livrable.

10.3 Phase 3 : Agent SLIDES (Semaines 7-9)

La troisieme phase etend la plateforme aux presentations. La Semaine 7 construit le DAG de presentation : DAG Builder pour slides, criteres Reflection orientes design, modeles optimises pour le contenu visuel, et templates de base (corporate, pitch, academique). La Semaine 8 implemente le pipeline de sortie : generateur PPTX via python-pptx, generateur PDF via ReportLab, mise en page intelligente avec grilles, contrastes et typographie, et export multi-format. La Semaine 9 finalise avec le polish et l'integration : preview dans le dashboard, edition post-generation, templates personnalisables. L'Agent SLIDES est le deuxieme MVP livrable.

10.4 Phase 4 : Agents DOC + DATA + RECHERCHE (Semaines 10-14)

La quatrieme phase deploie trois agents supplementaires en utilisant le kernel existant. Les Semaines 10-11 construisent l'Agent DOC avec son DAG Outline-Sections-Content-PDF, ses criteres de reflexion (Structure 30%, Profondeur 25%, Style 20%), et la generation PDF/DOCX professionnelle. Les Semaines 12-13 implementent l'Agent DATA avec son DAG Schema-Query-Process-Visual, un sandbox Python dedie (pandas, matplotlib, seaborn), et des criteres de reflexion orientes precision (Precision 35%, Visualisation 25%, Insight 20%). La Semaine 14 ajoute l'Agent RECHERCHE avec integration web-search et academic-search, et des criteres de reflexion orientes fiabilite (Pertinence 30%, Exhaustivite 25%, Fiabilite 25%).

10.5 Phase 5 : Scale + EMAIL + MARKETING (Semaines 15-18)

La cinquieme et derniere phase optimise la plateforme pour la production a grande echelle et ajoute les deux agents restants. Les Semaines 15-16 sont consacrees a l'optimisation scale : routeur RL entraine sur donnees reelles, cache predictif qui anticipe les requetes, load balancing multi-region, monitoring et alerting, et optimisation des couts visant une reduction de 40 a 60%. Les Semaines 17-18 ajoutent l'Agent EMAIL (MoA leger, A/B testing, personnalisation) et l'Agent MARKETING (copy, funnels, SEO, analytics). A la fin de cette phase, la plateforme AgentForge est complete avec ses 7 agents et une infrastructure de production eprouvee.

Phase	Duree	Contenu	Livrable
1 - Fondations	3 sem	MoA Kernel + Reflection + Infrastructure	Kernel operationnel
2 - Agent DEV	3 sem	DAG Code + Sandbox + Deployer	MVP Developpeur
3 - Agent SLIDES	3 sem	DAG Slides + Output Pipeline + Templates	MVP SLIDES

4 - DOC/DATA/RCHG	5 sem	3 agents supplementaires + DAGs specifiques	5 agents actifs
5 - Scale + Email/Mkg	4 sem	Optimisation + 2 agents finaux	Plateforme complete

Tableau 9 : Feuille de route en 5 phases

11. Estimation des Couts et Modele Economique

11.1 Couts de Construction

L'estimation des couts de construction se decompose en deux categories principales : les couts d'infrastructure (serveurs, bases de donnees, stockage) et les couts LLM (tokens consommes pendant le developpement et les tests). Durant la Phase 1, les couts LLM sont modestes (~\$200/mois) car le systeme est en developpement avec des tests limites. L'infrastructure coute environ \$50/mois (un petit VPS + PostgreSQL manage). Les phases suivantes voient les couts LLM augmenter a mesure que les agents sont testes et affines, tandis que l'infrastructure evolue progressivement.

Phase	Duree	Couts LLM/mois	Infra/mois	Total phase
1 - Fondations	3 sem	\$200	\$50	\$750
2 - Agent DEV	3 sem	\$500	\$100	\$1 500
3 - Agent SLIDES	3 sem	\$400	\$100	\$1 200
4 - DOC/DATA/RCHG	5 sem	\$800	\$200	\$2 500
5 - Scale + reste	4 sem	\$600	\$300	\$2 200
TOTAL	18 sem			\$8 150

Tableau 10 : Estimation des couts de construction

11.2 Modele Economique et Rentabilite

Le modele economique repose sur une structure d'abonnement a 4 niveaux (Gratuit, Plus a \$29/mois, Pro a \$99/mois, Entreprise a \$299/mois) avec une marge brute estimee a 60-70% sur les plans payants. Le cout moyen par generation est de \$0.02 a \$0.10 selon la complexite de la tache, ce qui permet une forte marge meme sur le plan Pro. Le point mort est estime a 150 abonnees Pro, soit environ \$15 000 de

revenus mensuels. Avec une strategie d'acquisition agressive (freemium + referral), ce seuil peut etre atteint en 3 a 6 mois apres le lancement. Le facteur cle de rentabilite est le taux de hit du cache : chaque requete servie depuis le cache coute \$0 en tokens LLM, ce qui ameliore directement la marge brute.

12. Les 12 Mecanismes d'Orchestration

12.1 Vue d'Ensemble des 12 Mecanismes

Ce document a identifie et documente en detail 12 mecanismes d'orchestration distincts qui constituent l'ADN d'AgentForge. Ces mecanismes ne sont pas de simples fonctions isolees : ils s'articulent les uns avec les autres pour former un systeme coherent ou chaque composant renforce les autres. Le MoA distribue les taches, le DAG structure la generation, le Reflection Agent garantit la qualite, l'Auto-Fix corrige les erreurs, le Cost Optimizer equilibre les couts, le Cache Manager accelere les reponses, le Sandbox isole les executions, le Deployer automatise les mises en production, et la securite multicouche protege l'ensemble.

#	Mecanisme	Categorie	Impact
1	Mixture-of-Agents (9 LLMs)	Orchestration noyau	Critique
2	DAG-Based Code Generation	Orchestration noyau	Critique
3	Reflection Agent (Vote multi-criteres)	Orchestration noyau	Critique
4	3-Level Auto-Fix Engine	Orchestration noyau	Eleve
5	RL-Optimized Model Router	Routage/Optimisation	Eleve
6	Cost Optimizer Cascade	Routage/Optimisation	Moyen
7	Cache Orchestration L1+L2	Routage/Optimisation	Eleve
8	Sandbox Warm Pool	Execution/Contexte	Eleve
9	Context-Aware Generation	Execution/Contexte	Critique
10	Linear Context Management	Execution/Contexte	Eleve
11	Cloudflare Edge Deployer	Deploiement/Securite	Moyen
12	Multi-Layer Security Orchestration	Deploiement/Securite	Critique

Tableau 11 : Les 12 mecanismes d'orchestration

12.2 Interactions entre Mechanismes

Les mecanismes ne fonctionnent pas en isolation. Le flux typique d'une requete illustre leurs interactions. Quand un utilisateur soumet un prompt, le Cache Manager (7) verifie si une reponse similaire existe deja. Si non, le Cost Optimizer (6) et le Routeur RL (5) selectionnent les modeles optimaux. Le MoA (1) decompose la tache et distribue les sous-taches. Le DAG (2) structure la generation en niveaux. Le Context-Aware (9) et le Linear Context (10) nourrissent chaque generation avec les dependances adequates. Les resultats sont evalues par le Reflection Agent (3). Si des erreurs sont detectees, l'Auto-Fix (4) les corrige en cascade. Si du code doit etre execute, le Sandbox (8) l'isole. Si le resultat est une application, le Deployer (11) la met en production. Et pendant tout le processus, la Security Orchestration (12) protege chaque echange. Cette choregraphie est le veritable "secret" d'AgentForge : ce n'est pas un mecanisme unique qui fait la difference, mais l'integration harmonieuse de douze mecanismes.

13. Les 25 Commandements de l'Architecture Agent

Cette section synthetise les principes fondamentaux issus de l'analyse des deux documents sources (le Compilateur d'Architecture Intelligente et le Guide de Clonage Genspark). Ces commandements sont des regles non negociables qui doivent guider chaque decision d'implementation de la plateforme AgentForge.

1. Un kernel, plusieurs configurations. Ne jamais dupliquer le code d'orchestration entre les agents.
2. Le Reflection Agent est non negociable. Sans vote multi-criteres, la qualite est aleatoire.
3. Toujours generer par niveaux de dependances (DAG). Jamais sequentiellement.
4. Le cache est une fonctionnalite, pas une optimisation. Le concevoir des le premier jour.
5. Le routeur RL s'ameliore avec le temps. Collecter les donnees d'entrainement des le lancement.
6. L'Auto-Fix en cascade economise des tokens. Toujours tenter le pattern avant le LLM.
7. Le contexte lineaire evite les conflits. Ne jamais injecter tout le code genere dans chaque fichier.
8. Le sandbox est obligatoire pour tout code execute. Aucune exception.
9. La securite est multicouche. Jamais un seul mecanisme de protection.
10. Le deploiement doit etre automatise et reproductible. Pas de deploiement manuel.
11. Mesurer tout : couts, temps, qualite. Les donnees pilotent les optimisations.
12. Le seuil de confiance 0.95 est un minimum. Ne jamais livrer en dessous.
13. Les modeles open-source sont des reserves de cout. Les utiliser pour les taches simples.

- 14. La gestion du contexte est plus importante que le choix du modele.
- 15. Toujours privilegier la qualite percue par l'utilisateur sur la qualite technique interne.
- 16. Le Template Learning est un avantage concurrentiel. Les patterns reutilises s'amelior.
- 17. L'orchestration parallele est toujours superieure a la sequentielle.
- 18. Le Cost Optimizer doit etre transparent. L'utilisateur doit comprendre les couts.
- 19. Le pool de sandbox prechauffes est indispensable pour la reactivite.
- 20. L'invalidation du cache doit etre granulaire. Jamais de flush complet.
- 21. Les criteres de reflexion doivent etre adaptes par agent. Pas de critere universel.
- 22. Le RL Router doit avoir une strategie epsilon-greedy pour explorer de nouveaux modeles.
- 23. La configuration d'agent est du code, pas des donnees. La versionner avec Git.
- 24. Toujours prevoir un fallback quand un modele est indisponible. Jamais de point de defaillance unique.
- 25. L'objectif final est l'experience utilisateur. Tous les mecanismes servent cet objectif.

14. Prochaines Etapes et Recommandations

14.1 Actions Immmdiates

Pour lancer la construction d'AgentForge, trois actions doivent etre entreprises immediatement. La premiere est la mise en place de l'environnement de developpement : un depot Git monorepo avec Turborepo, les configurations Docker Compose pour PostgreSQL 16 et Redis 7, et le scaffolding des packages (api, web, shared, sandbox). La deuxieme action est l'obtention des cles API pour les 9 modeles LLM : les fournisseurs principaux (OpenAI, Anthropic, Google, DeepSeek, Mistral) offrent des credits gratuits pour les nouveaux comptes, ce qui permet de commencer les tests sans investissement initial significatif. La troisieme action est la definition precise des DAGs pour les deux premiers agents (Developpeur et SLIDES), en s'appuyant sur les schemas presentes dans ce document.

14.2 Risques et Mitigations

Risque	Probabilite	Impact	Mitigation
Dependance aux fournisseurs LLM	Elevee	Critique	Routeur RL + fallback multi-modeles + modeles open-source en reserve

Depassement des couts LLM	Moyenne	Eleve	Cache agressif + Cost Optimizer + alertes budget + modeles legeres pour taches simples
Qualite insuffisante des sorties	Faible	Critique	Reflection Agent a seuil 0.95 + Auto-Fix 3 niveaux + feedback utilisateur
Faible de securite dans le sandbox	Faible	Critique	Isolation reseau + liste noire de commandes + limites ressources + audit regulier
Scalabilite insuffisante	Moyenne	Moyen	Architecture horizontale + cache Redis + load balancing + monitoring

Tableau 12 : Risques et mitigations

14.3 Indicateurs de Succes

Le succes de la plateforme sera mesure par cinq indicateurs cles. Le premier est le **taux de confiance moyen** des generations : la cible est superieure a 0.92 en moyenne, et superieure a 0.95 pour l'Agent Developpeur. Le deuxieme est le **taux de hit du cache** : la cible est superieure a 30% en production. Le troisieme est le **temps de reponse median** : la cible est inferieure a 15 secondes du prompt au resultat pour les generations simples, et inferieure a 60 secondes pour les projets complets avec deploiement. Le quatrieme est le **cout moyen par generation** : la cible est inferieure a \$0.05. Le cinquieme est le **taux de satisfaction utilisateur** : la cible est superieure a 85% de reponses positives.